WEEK 3

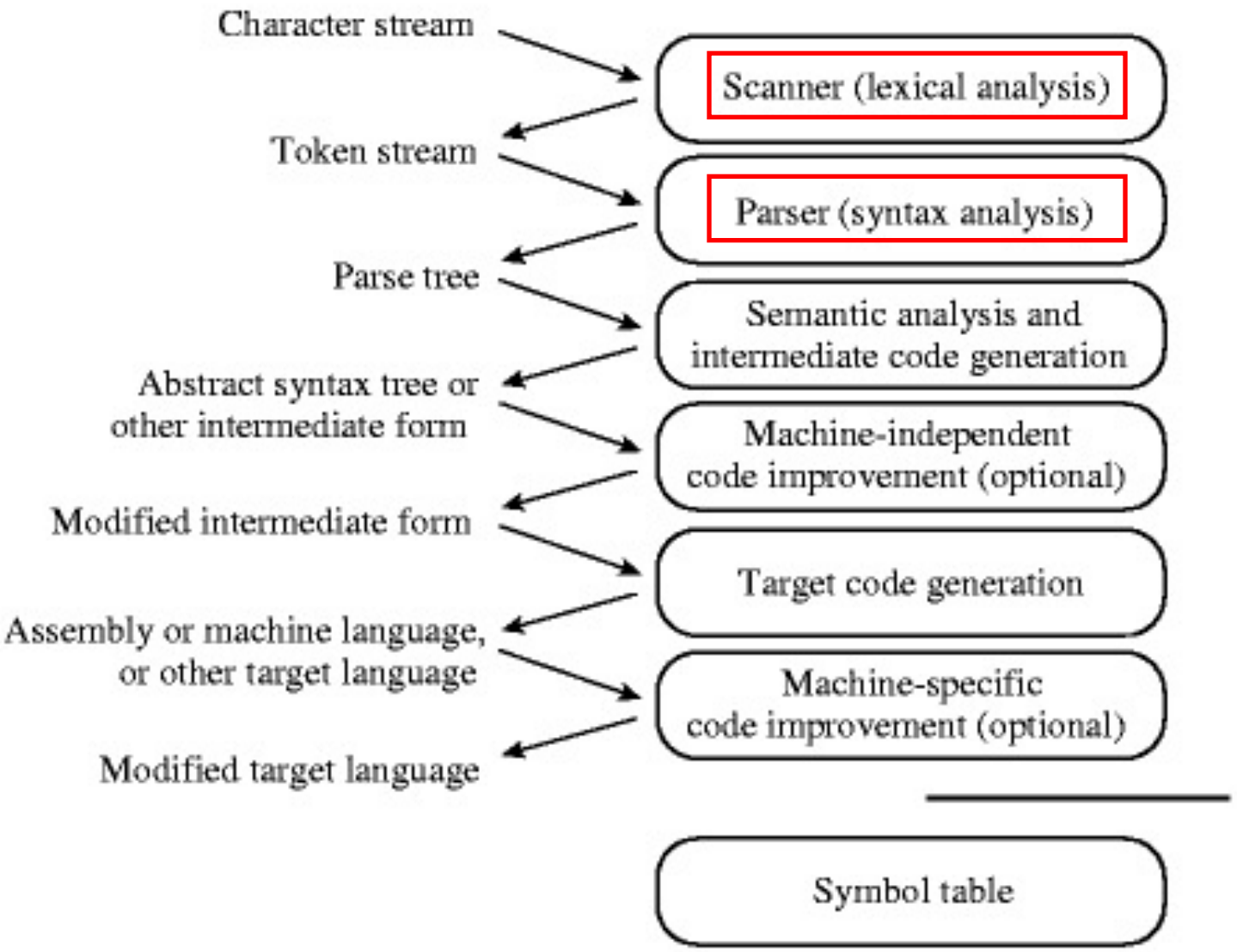
LEXICAL ANALYSIS

Introduction

- Language implementation systems must analyze source code, regardless of the specific implementation approach

- Nearly all syntax analysis is based on a formal description of the syntax of the source language (BNF)

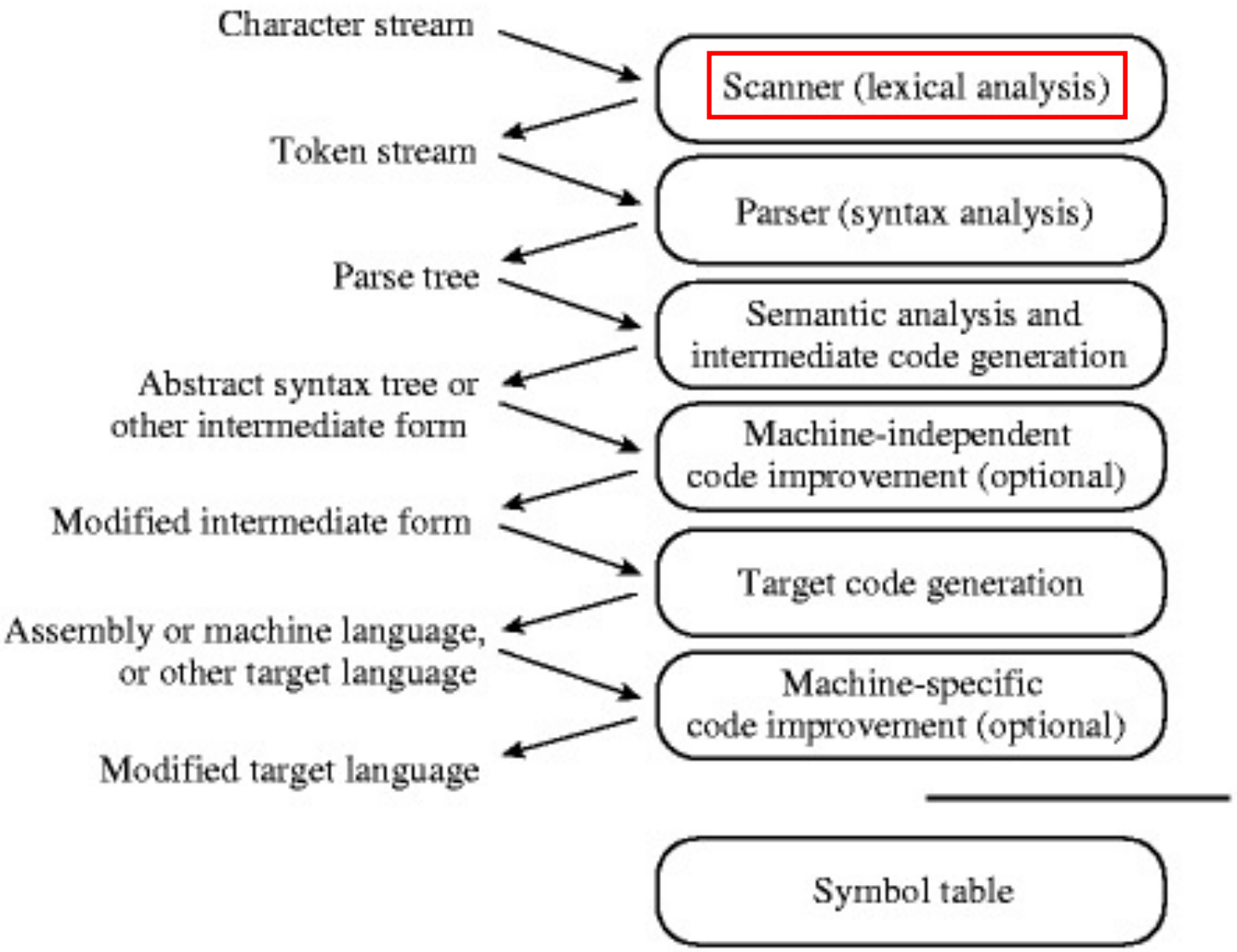
Phases of Compilation



Syntax Analysis

- The syntax analysis portion of a language processor nearly always consists of two parts:
  - A low-level part called a *lexical analyzer* (mathematically, a finite automaton based on a regular grammar) – *deals with names, numbers*
  - A high-level part called a *syntax analyzer*, or parser (mathematically, a push-down automaton based on a context-free grammar, or BNF) – *deals with expressions, statements, program units*

Phases of Compilation



Lexical Analysis

- A lexical analyzer is a pattern matcher for character strings
- A lexical analyzer is a "front-end" for the parser
- Identifies substrings of the source program that belong together - *lexemes*
  - Lexemes match a character pattern, which is associated with a lexical category called a *token*
  - `sum` is a lexeme; its token may be `IDENT`
- Tokens of a programming language are a regular language, lexical analyzer is a finite automaton

Lexical analyzers (Scanners)

- Scanners
  - Task: recognize language tokens
  - Implementation: Deterministic Finite Automaton
    - Transition based on the next character

Lexical Analysis

- Tokens for a calculator language:

```
assign → :=
plus   → +
minus  → -
times  → *
div    → /
lparen → (
rparen → )
id     → letter ( letter | digit )*
number → digit digit * | digit * ( . digit | digit . ) digit *
comment → /* ( non* | * non/ )* (*)+ /
          | // ( nonnewline )* newline
```

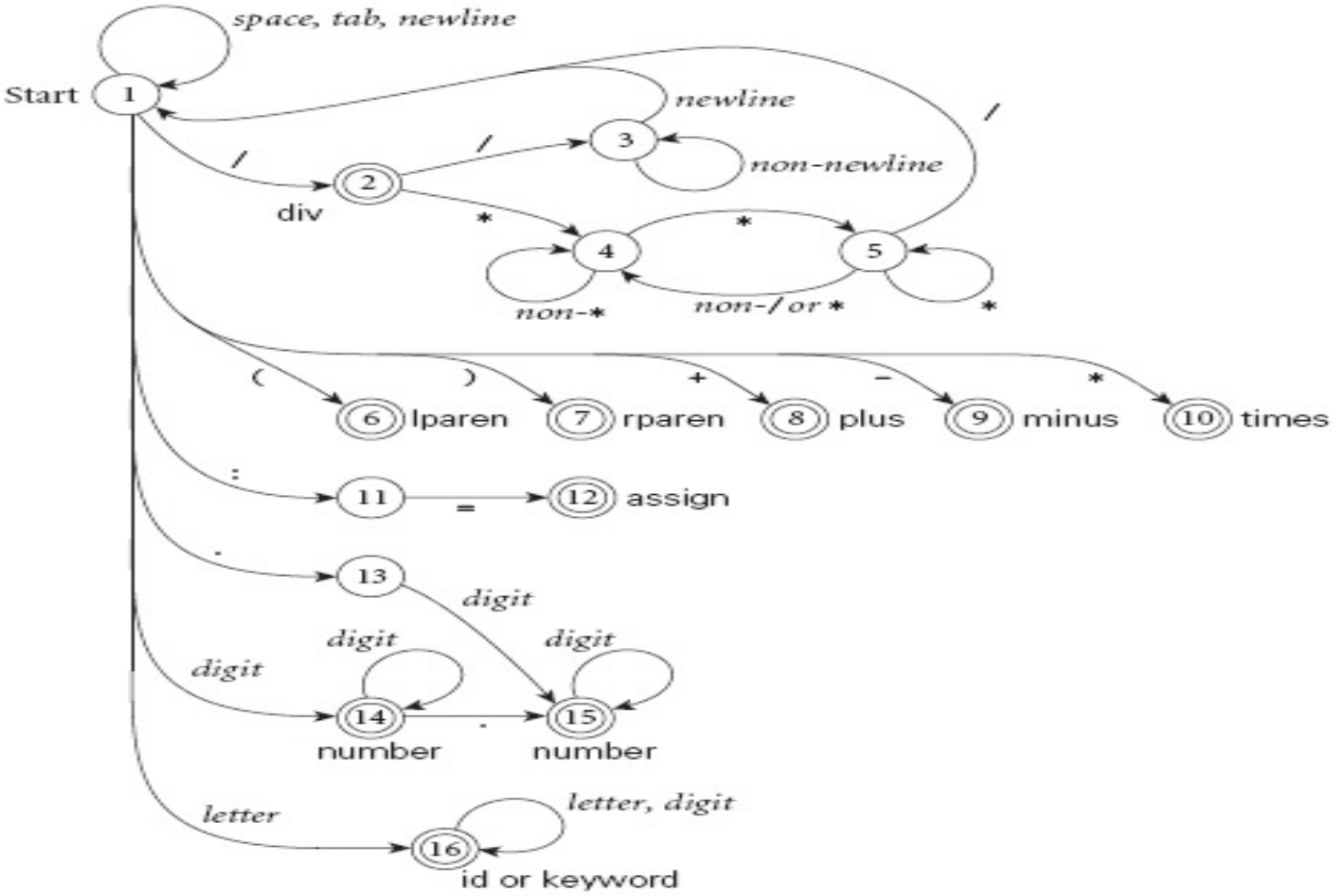
(except for read and write)

Lexical Analysis

- Suppose we are building a scanner:
  - We read the characters one at a time with look-ahead
  - If it is one of the one-character tokens ( ) + - *
    we announce that token
  - If it is a /, we look at the next character
    - if that is not a * or /, we announce / and reuse the look-ahead
  - If it is a :, we look at the next character
    - if that is a = we announce :=
  - If it is a letter, we keep reading letters and digits and maybe underscores until we can't anymore
    - then we check to see if it is a reserved word
  - If it is a digit, we keep reading until we find a non-digit
    - if non-digit is not a . we announce an integer
    - otherwise, we keep looking for a real number
    - if the character after the . is not a digit we announce an integer and reuse the . and the look-ahead

State Diagram Design

- Pictorial representation of a scanner for calculator tokens, in the form of a finite automaton



Lexical Analysis (continued)

- In many cases, transitions can be combined to simplify the state diagram
  - When recognizing an identifier, all uppercase and lowercase letters are equivalent
    - Use a character class that includes all letters
  - When recognizing an integer literal, all digits are equivalent – use a digit class

Lexical Analysis (continued)

- Reserved words and identifiers can be recognized together (rather than having a part of the diagram for each reserved word)
  - Use a lookup table to determine whether a possible identifier is in fact a reserved word

Lexical Analysis (continued)

- Lexical analyzers tend to be built three ways
  - ad-hoc
  - semi-mechanical pure DFA
    (usually realized as nested case statements)
  - table-driven DFA

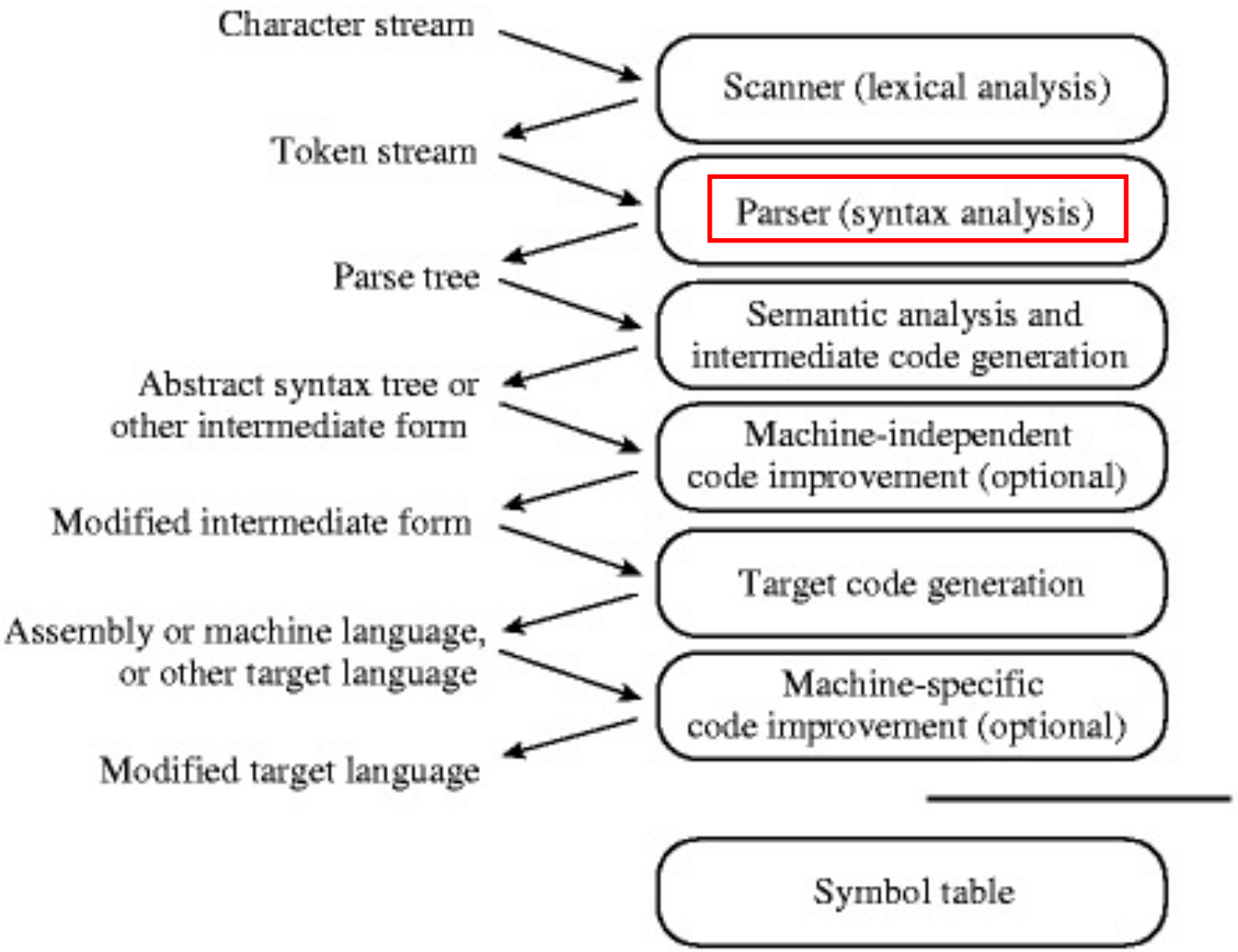
- Ad-hoc generally yields the fastest, most compact code by doing lots of special-purpose things

Lexical Analysis (continued)

- Writing a pure DFA as a set of nested case statements is a surprisingly useful programming technique

- Table-driven DFA is what `lex` and `scangen` produce
  - lex (flex) in the form of C code
  - scangen in the form of numeric tables and a separate driver

Phases of Compilation



Parsing (Syntax Analysis)

- Goals of the parser, given an input program:
  - Find all syntax errors; for each, produce an appropriate diagnostic message and recover quickly
  - Produce the parse tree, or at least a trace of the parse tree, for the program

Parsers

- Parsers
  - Task: recognize language syntax (organization of tokens)
  - Implementation:
    - Top-down parsing
    - Bottom-up parsing

Parsing (continued)

- Two categories of parsers
  - *Top down* – produce the parse tree, beginning at the root
    - Order is that of a leftmost derivation
    - Traces or builds the parse tree in preorder
  - *Bottom up* – produce the parse tree, beginning at the leaves
    - Order is that of the reverse of a rightmost derivation
- Useful parsers look only one token ahead in the input

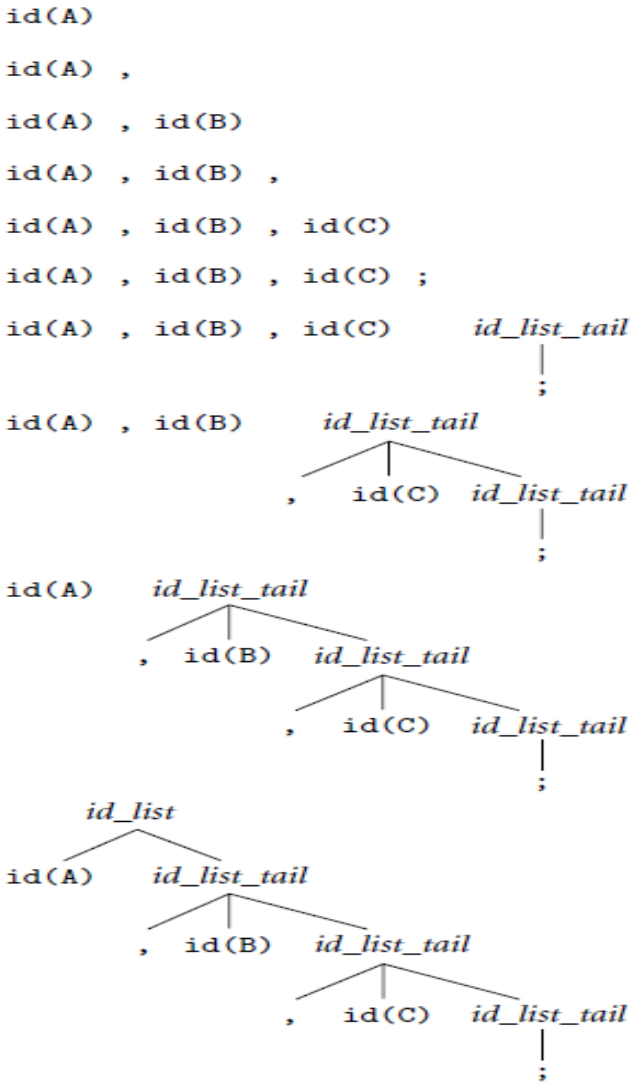
Bottom-up Parsers

- LR parsers
  - Left-to-right scan of input, Rightmost derivation
  - Construct the parse tree from the leaves up, recognizing when a collection of leaves or other nodes can be joined together as the children of a single parent

id_list ⟶ id *id_list_tail*

id_list_tail ⟶ , id *id_list_tail*

id_list_tail ⟶ ;



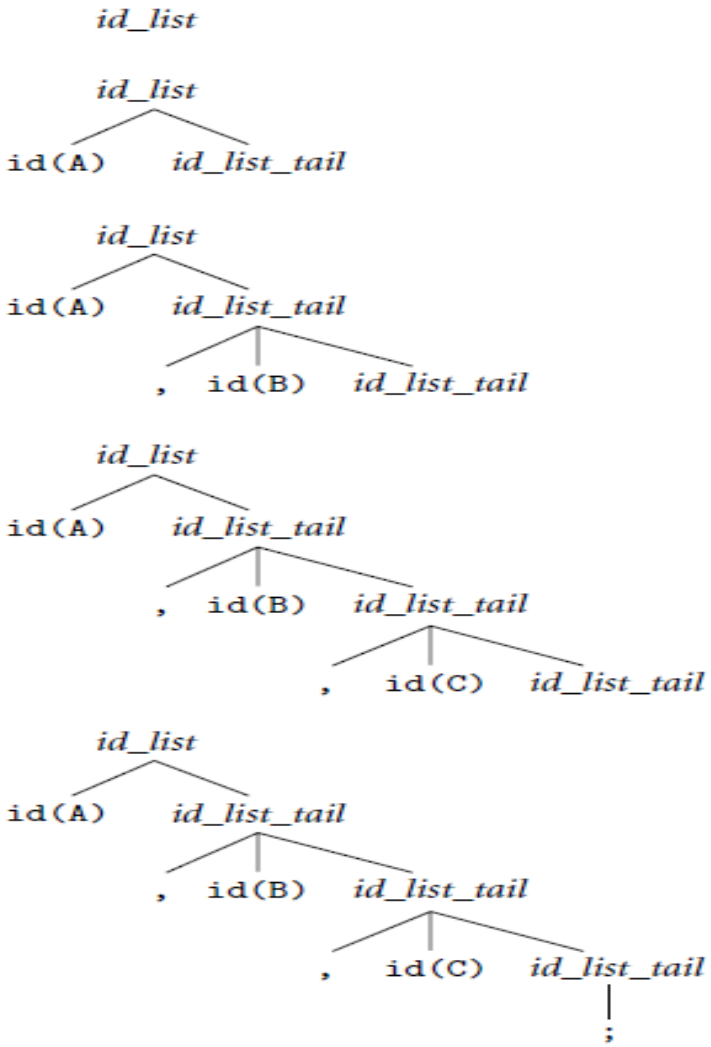
Top-down Parsers

- Recursive descent parsers
- LL parsers
  - Left-to-right scan of input, Leftmost derivation
  - Construct the parse tree from the root down, predicting at each step which production will be used to expand the current node, based on the next available token of input

id_list ⟶ id *id_list_tail*

id_list_tail ⟶ , id *id_list_tail*

id_list_tail ⟶ ;



Recursive-Descent Parsing

- There is a subprogram for each nonterminal in the grammar, which can parse sentences that can be generated by that nonterminal

- EBNF is ideally suited for being the basis for a recursive-descent parser, because EBNF minimizes the number of nonterminals

Recursive-Descent Parsing (continued)

- A grammar for simple expressions:

```
<expr> → <term> {(+ | -) <term>}
<term> → <factor> {(* | /) <factor>}
<factor> → id | int_constant | ( <expr> )
```

Recursive-Descent Parsing (continued)

- Assume we have a lexical analyzer named `lex`, which puts the next token code in `nextToken`
- The coding process when there is only one RHS:
  - For each terminal symbol in the RHS, compare it with the next input token; if they match, continue, else there is an error
  - For each nonterminal symbol in the RHS, call its associated parsing subprogram

Recursive-Descent Parsing (continued)

```
/* Function expr
   Parses strings in the language
   generated by the rule:
   <expr> → <term> {(+ | -) <term>}
 */

void expr() {

/* Parse the first term */

  term();
/* As long as the next token is + or -, call
   lex to get the next token and parse the
   next term */

  while (nextToken == ADD_OP || nextToken == SUB_OP){
    lex();
    term();
  }
}
```

- This particular routine does not detect errors
- Convention: Every parsing routine leaves the next token in **`nextToken`**

Recursive-Descent Parsing (continued)

- A nonterminal that has more than one RHS requires an initial process to determine which RHS it is to parse
  - The correct RHS is chosen on the basis of the next token of input (the lookahead)
  - The next token is compared with the first token that can be generated by each RHS until a match is found
  - If no match is found, it is a syntax error

Recursive-Descent Parsing (continued)

```
/* term
Parses strings in the language generated by the rule:
<term> -> <factor> {(* | /) <factor>)
*/
void term() {

/* Parse the first factor */
  factor();

/* As long as the next token is * or /,
   next token and parse the next factor */
  while (nextToken == MULT_OP || nextToken == DIV_OP) {
    lex();
    factor();
  }
} /* End of function term */
```

Recursive-Descent Parsing (continued)

```
/* Function factor
   Parses strings in the language
   generated by the rule:
   <factor> -> id | int_constant |  (<expr>)  */

 void factor() {

 /* Determine which RHS */
   if (nextToken == ID_CODE || nextToken == INT_CODE)

 /* For the RHS id or int, just call lex */
     lex();

/* If the RHS is (<expr>) - call lex to pass over the left parenthesis,
   call expr, and check for the right parenthesis */
  else if (nextToken == LP_CODE) {
     lex();
     expr();
     if (nextToken == RP_CODE)
        lex();
     else
        error();
    }  /* End of else if (nextToken == LP_CODE */

   else error(); /* Neither RHS matches */
}
```

Recursive-Descent Parsing (continued)

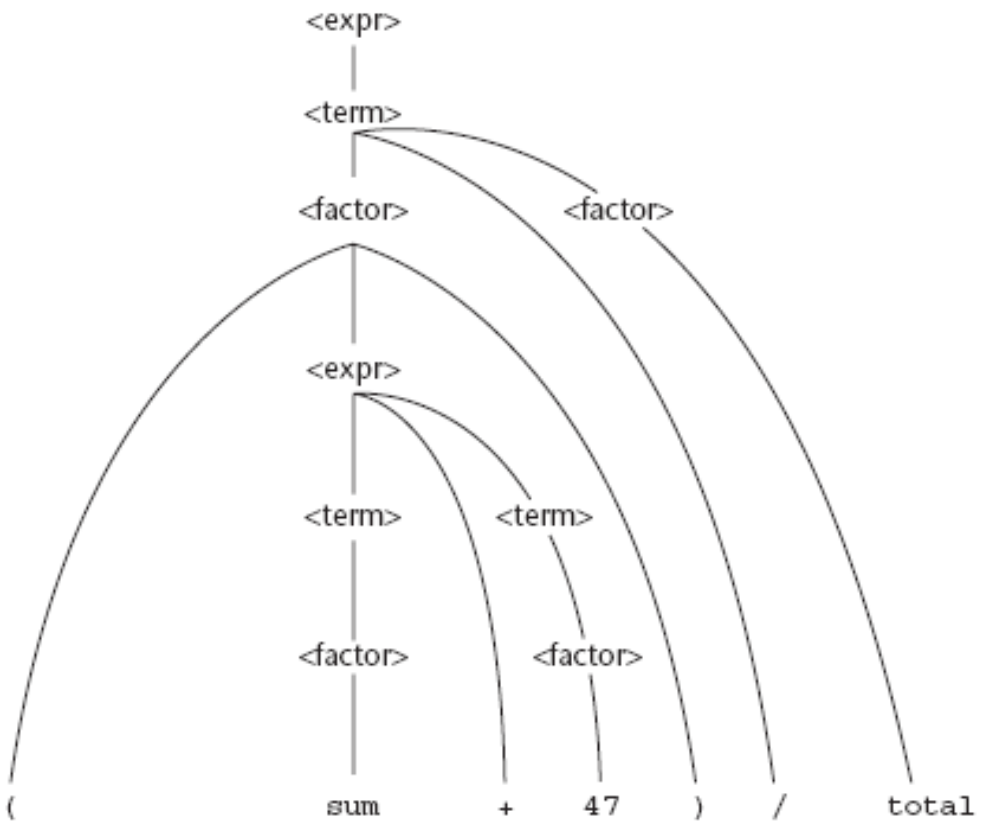
- Trace of the lexical and syntax analyzers on `(sum + 47) / total`

```
Next token is: 25 Next lexeme is (
Enter <expr>
Enter <term>
Enter <factor>
Next token is: 11 Next lexeme is sum
Enter <expr>
Enter <term>
Enter <factor>
Next token is: 21 Next lexeme is +
Exit <factor>
Exit <term>
Next token is: 10 Next lexeme is 47
Enter <term>
Enter <factor>
Next token is: 26 Next lexeme is )
Exit <factor>
Exit <term>
Exit <expr>
Next token is: 24 Next lexeme is /
Exit <factor>
Next token is: 11 Next lexeme is total
Enter <factor>
Next token is: -1 Next lexeme is EOF
Exit <factor>
Exit <term>
Exit <expr>
```

Parse Tree for (sum+47)/total

Figure 4.2

Parse tree for
(sum + 47) / total



LL Parsers

- An LL grammar for a simple calculator language:

```
 1. program  → stmt_list $$
 2. stmt_list→ stmt stmt_list
 3.            | ε
 4. stmt     → id := expr
 5.            | read id
 6.            | write expr
 7. expr     → term term_tail
 8. term_tail→ add_op term term_tail
 9.            | ε
10. term     → fact fact_tail
11. fact_tail→ mult_op fact fact_tail
12.            | ε
13. fact     → ( expr )
14.            | id
15.            | number
16. add_op   → +
17.            | -
18. mult_op  → *
19.            | /
```

LL Parsers

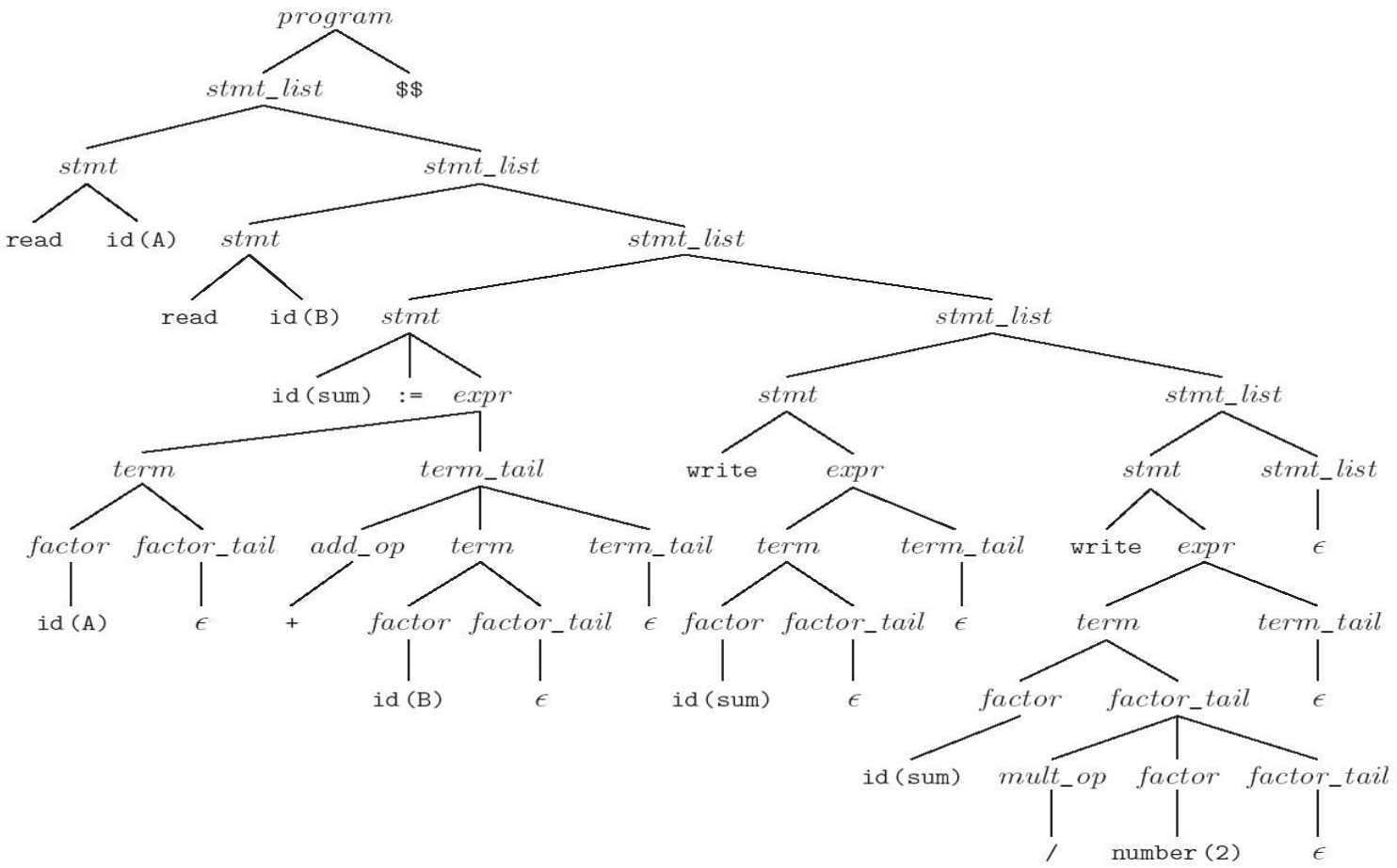
- We parse a string with this grammar by building the parse tree incrementally
- Example (average program)

```
read A
read B
sum := A + B
write sum
write sum / 2
```

- We start at the top and predict needed productions on the basis of the current left-most non-terminal in the tree and the current input token

LL Parsers

- Parse tree for the average program



LL Parse Table

- Table-driven LL parsing: you have a big loop in which you repeatedly look up an action in a two-dimensional table based on current leftmost non-terminal and current input token. The actions are

  (1) match a terminal

  (2) predict a production

  (3) announce a syntax error

LL Parse Table

- LL(1) parse table for parsing for calculator language

Top-of-stack nonterminal	Current input token											
	id	number	read	write	:=	(	)	+	-	*	/	$$
program	1	–	1	1	–	–	–	–	–	–	–	1
stmt_list	2	–	2	2	–	–	–	–	–	–	–	3
stmt	4	–	5	6	–	–	–	–	–	–	–	–
expr	7	7	–	–	–	7	–	–	–	–	–	–
term_tail	9	–	9	9	–	–	9	8	8	–	–	9
term	10	10	–	–	–	10	–	–	–	–	–	–
factor_tail	12	–	12	12	–	–	12	12	12	11	11	12
factor	14	15	–	–	–	13	–	–	–	–	–	–
add_op	–	–	–	–	–	–	–	16	17	–	–	–
mult_op	–	–	–	–	–	–	–	–	–	18	19	–

LL Parse Table

- To keep track of the left-most non-terminal, you push the as-yet-unseen portions of productions onto a stack

- The key thing to keep in mind is that the stack contains all the stuff you expect to see between now and the end of the program

Trace of table-driven LL parse of average program

Parse stack	Input stream	Comment
program	`read A read B ...`	initial stack contents
stmt_list $$	`read A read B ...`	predict *program* ⟶ *stmt_list* $$
stmt stmt_list $$	`read A read B ...`	predict *stmt_list* ⟶ *stmt stmt_list*
`read id` *stmt_list* $$	`read A read B ...`	predict *stmt* ⟶ `read id`
`id` *stmt_list* $$	`A read B ...`	match `read`
stmt_list $$	`read B sum := ...`	match `id`
stmt stmt_list $$	`read B sum := ...`	predict *stmt_list* ⟶ *stmt stmt_list*
`read id` *stmt_list* $$	`read B sum := ...`	predict *stmt* ⟶ `read id`
`id` *stmt_list* $$	`B sum := ...`	match `read`
stmt_list $$	`sum := A + B ...`	match `id`
stmt stmt_list $$	`sum := A + B ...`	predict *stmt_list* ⟶ *stmt stmt_list*
`id :=` *expr stmt_list* $$	`sum := A + B ...`	predict *stmt* ⟶ `id :=` *expr*
`:=` *expr stmt_list* $$	`:= A + B ...`	match `id`
expr stmt_list $$	`A + B ...`	match `:=`
term term_tail stmt_list $$	`A + B ...`	predict *expr* ⟶ *term term_tail*
factor factor_tail term_tail stmt_list $$	`A + B ...`	predict *term* ⟶ *factor factor_tail*
`id` *factor_tail term_tail stmt_list* $$	`A + B ...`	predict *factor* ⟶ `id`
factor_tail term_tail stmt_list $$	`+ B write sum ...`	match `id`
term_tail stmt_list $$	`+ B write sum ...`	predict *factor_tail* ⟶ ϵ
add_op term term_tail stmt_list $$	`+ B write sum ...`	predict *term_tail* ⟶ *add_op term term_tail*
`+` *term term_tail stmt_list* $$	`+ B write sum ...`	predict *add_op* ⟶ `+`
term term_tail stmt_list $$	`B write sum ...`	match `+`
factor factor_tail term_tail stmt_list $$	`B write sum ...`	predict *term* ⟶ *factor factor_tail*
`id` *factor_tail term_tail stmt_list* $$	`B write sum ...`	predict *factor* ⟶ `id`
factor_tail term_tail stmt_list $$	`write sum ...`	match `id`
term_tail stmt_list $$	`write sum write ...`	predict *factor_tail* ⟶ ϵ
stmt_list $$	`write sum write ...`	predict *term_tail* ⟶ ϵ
stmt stmt_list $$	`write sum write ...`	predict *stmt_list* ⟶ *stmt stmt_list*
`write` *expr stmt_list* $$	`write sum write ...`	predict *stmt* ⟶ `write` *expr*
expr stmt_list $$	`sum write sum / 2`	match `write`
term term_tail stmt_list $$	`sum write sum / 2`	predict *expr* ⟶ *term term_tail*
factor factor_tail term_tail stmt_list $$	`sum write sum / 2`	predict *term* ⟶ *factor factor_tail*
`id` *factor_tail term_tail stmt_list* $$	`sum write sum / 2`	predict *factor* ⟶ `id`
factor_tail term_tail stmt_list $$	`write sum / 2`	match `id`
term_tail stmt_list $$	`write sum / 2`	predict *factor_tail* ⟶ ϵ

LL Grammar Class

- The Left Recursion Problem
  - If a grammar has left recursion, either direct or indirect, it cannot be the basis for a top-down parser
    - A grammar can be modified to remove direct left recursion as follows:

      For each nonterminal, A,

      1. Group the A-rules as $A \rightarrow A\alpha_1 \mid \ldots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \ldots \mid \beta_n$ where none of the β's begins with A
      2. Replace the original A-rules with

         $A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \ldots \mid \beta_n A'$

         $A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \ldots \mid \alpha_m A' \mid \epsilon$

Eliminating Left Recursion

- Grammar: $E \rightarrow E + T \mid T$
  $T \rightarrow T * F \mid F$
  $F \rightarrow (E) \mid id$

  - for the E-rules, we have $\alpha 1 = + T$ and $\beta = T$, so
    $E \rightarrow T\ E'$
    $E' \rightarrow + T\ E' \mid \varepsilon$
  - for the T-rules, we have $\alpha_1 = * F$ and $\beta = F$, so
    $T \rightarrow F\ T'$
    $T' \rightarrow * F\ T' \mid \varepsilon$
  - no left recursion for F–rules

Eliminating Left Recursion

- Complete replacement grammar:

  $E \rightarrow T\ E'$

  $E' \rightarrow +\ T\ E' \mid \varepsilon$

  $T \rightarrow F\ T'$

  $T' \rightarrow *\ F\ T' \mid \varepsilon$

  $F \rightarrow (E) \mid id$